```
'''
  1. File Handling
  2. Exception Handling
'''


## File Handling

'''
  Focus: Reading and writing for logging, processing sensor data an
  Intro:
    * File handling helps to interact with external files..
    * Why? Store Temp/pressure/speed senor data, writing daily repo
  Modes:
    * "r": Read Only
    * "w": Write (Overwrite)
    * "a": Append (adds to the file)
    * "r+": Read & write

  Method:
    * read(): reads entire file
    * readlines(): reads line by line into a list
    * readline: read line by line if file is large
'''


## Open and close file


import os
os.listdir(".")
```

```
['.config', 'sample_data']
```

```
file = open("log.txt","w")
file.write("Machine Started. This is our 1st experiment!")
file.close()
```

```
import os
os.listdir(".")
```

```
['.config', 'log.txt', 'sample_data']
```

Reading from a File

```
file = open("log.txt","r")
content = file.read()
print(content)
file.close()
```

```
This article is about the country. For other uses, see Unite
"UK" redirects here. For other uses, see UK (disambiguation)
United Kingdom of Great Britain and Northern Ireland
A flag composed of a red cross edged in white and superimpos
Flag
Anthem: "God Save the King"[a]
Duration: 1 minute and 4 seconds.1:04
Coats of arms:

Used in relation to Scotland (right) and elsewhere (left)

Show globe
Show map of Europe
Show British Overseas Territories and Crown Dependencies
Show their exclusive economic zones
Show all
Capital
and largest city
London
51°30′N 0°7′W
National language       English
Regional and minority languages[b]
ScotsUlster ScotsWelshCornishScottish GaelicIrishBritish Sig
Ethnic groups (2021)[c][4][5][6]
83.0% White
8.6% Asian
3.7% Black
2.7% Mixed
2.0% other
Religion (2021)[c][7][8][6]
```

```
46.5% Christianity
37.8% no religion
6.0% Islam
1.6% Hinduism
0.8% Sikhism
0.4% Buddhism
0.4% Judaism
0.6% other
5.9% not stated
Demonym(s)
BritishBritonBrit (colloquial)
Government      Unitary parliamentary constitutional monarch
• Monarch
Charles III
• Prime Minister
Keir Starmer
Legislature     Parliament
• Upper house
House of Lords
• Lower house
House of Commons
Formation
• Laws in Wales Acts
1535 and 1542
• Union of the Crowns
24 March 1603
• Treaty of Union
```

```
type(content)
```

```
str
```

```
file = open("log.txt","r")
lines = file.readlines()
print(type(lines))
print(f"No of lines: {len(lines)}")
print(lines)
file.close()
```

```
<class 'list'>
No of lines: 114
['This article is about the country. For other uses, see Unite
```

```
lines[100]
```

```
'The UK has been inhabited continuously since the Neolithic.
In AD 43 the Roman conquest of Britain began; the Roman depar
ture was followed by Anglo-Saxon settlement. In 1066 the Norm
ans conquered England. With the end of the Wars of the Roses
the Kingdom of England stabilised and began to grow in power,
resulting by the 16th century in the annexation of Wales and
the establishment of the British Empire. Over the course of t
he 17th century the role of the British monarchy was reduced,
particularly as a result of the English Civil War. In 1707 th
```

```
## Write to file


file = open("machine_status.txt","w")
file.write("Machine A: Running\n")
file.write("Machine B: Idle\n")
file.close()


## Append to file


file = open("machine_status.txt","a")
file.write("Machine C: Faulty\n")
file.close()


## Use Case 1: Logging Machine data
```

```
machines = {
    "Machine1": "Running",
    "Machine2": "Idle",
    "Machine3": "Faulty"
}

with open("status_log.txt","w") as file:
  for machine,status in machines.items():
    file.write(f"{machine}: {status}\n")


## Use Case 2: Reading Sensor Data from CSV


with open("sensor_data.csv","r") as file:
  lines = file.readlines()
  # print(lines)
  for line in lines[1:]:
    timestamp, temp, pressure = line.split("\t")
    print(f"Timestamp: {timestamp}, Temperature: {temp}, Pressure
  file.close()
```

```
Timestamp: 7/6/2025 8:00, Temperature: 78.5, Pressure: 101.3

Timestamp: 7/6/2025 9:00, Temperature: 80.2, Pressure: 101

Timestamp: 7/6/2025 10:00, Temperature: 95.1, Pressure: 99.9
```

```
with open("log.txt","r") as file:
  first_line = file.readline()
  print(f"First Line: {first_line}")
```

```
First Line: This article is about the country. For other uses
```

```
import time
with open("log.txt","r") as file:
  line = file.readline()
  while line:
```

```
    print(f"Read:",line.strip())
    time.sleep(5)
    line=file.readline()
```

```
Read: This article is about the country. For other uses, see U
Read: "UK" redirects here. For other uses, see UK (disambiguat
Read: United Kingdom of Great Britain and Northern Ireland
Read: A flag composed of a red cross edged in white and super
Read: Flag
Read: Anthem: "God Save the King"[a]
Read: Duration: 1 minute and 4 seconds.1:04
Read: Coats of arms:
Read:
Read: Used in relation to Scotland (right) and elsewhere (left
Read:
Read: Show globe
Read: Show map of Europe
---------------------------------------------------------------
--------------
KeyboardInterrupt                         Traceback (most
recent call last)
/tmp/ipython-input-38-3990466257.py in <cell line: 0>()
      4   while line:
      5     print(f"Read:",line.strip())
----> 6     time.sleep(5)
      7     line=file.readline()

KeyboardInterrupt:
```

```
'''
Why try-except?: File I/O can fail if file not found, file is locl
Instead of your program crashing, try-except lets you handle these
'''
```

```python
a = 4
b = 2
try:
  c = a/b
  print(f"Value of a: {a}, Value of b: {b}, Value of a/b: {c}")
except Exception as e:
  print(f"Exception: {e}")
```

```
Value of a: 4, Value of b: 2, Value of a/b: 2.0
```

```python
a = 4
b = 2
c = a/b
print(f"Value of a: {a}, Value of b: {b}, Value of a/b: {c}")
```

```
Value of a: 4, Value of b: 2, Value of a/b: 2.0
```

```python
a = 4
b = 0
c = a/b
print(f"Value of a: {a}, Value of b: {b}, Value of a/b: {c}")
print("After code crash but used try-except")
```

```
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
/tmp/ipython-input-24-3519287317.py in <cell line: 0>()
      1 a = 4
      2 b = 0
----> 3 c = a/b
      4 print(f"Value of a: {a}, Value of b: {b}, Value of a/b: {c}")
      5 print("After code crash but used try-except")
```

```python
a = 4
b = 0
try:
  c = a/b
  print(f"Value of a: {a}, Value of b: {b}, Value of a/b: {c}")
```

```
except Exception as e:
  print(f"Exception: {e}")
print("After code crash but used try-except")
```

```
Exception: division by zero
After code crash but used try-except
```

Best Practices - Handle File not found

```
#read missing file
with open("missing_file.txt","r") as file:
  data = file.read()
  print(data)
  file.close()
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
/tmp/ipython-input-26-2576998585.py in <cell line: 0>()
      1 #read missing file
----> 2 with open("missing_file.txt","r") as file:
      3   data = file.read()
      4   print(data)
      5   file.close()

FileNotFoundError: [Errno 2] No such file or directory:
```

```
try:
  with open("missing_file.txt","r") as file:
    data = file.read()
    print(data)
    file.close()
except FileNotFoundError:
  print("File was not found")
```

```
File was not found
```

```
def read_data(filename):
    with open(filename,"r") as file:
```

```
      data = file.read()
      file.close()
    return data


## Validate File existence before reading


import os
# filename = "machine_status.txt"
filename = "machine_statusssjkhuie.txt"
status = os.path.exists(filename)
print(f"Status of file: {filename}: {status}")
```

```
Status of file: machine_statusssjkhuie.txt: False
```